



Full-stack software development

Custom Hooks

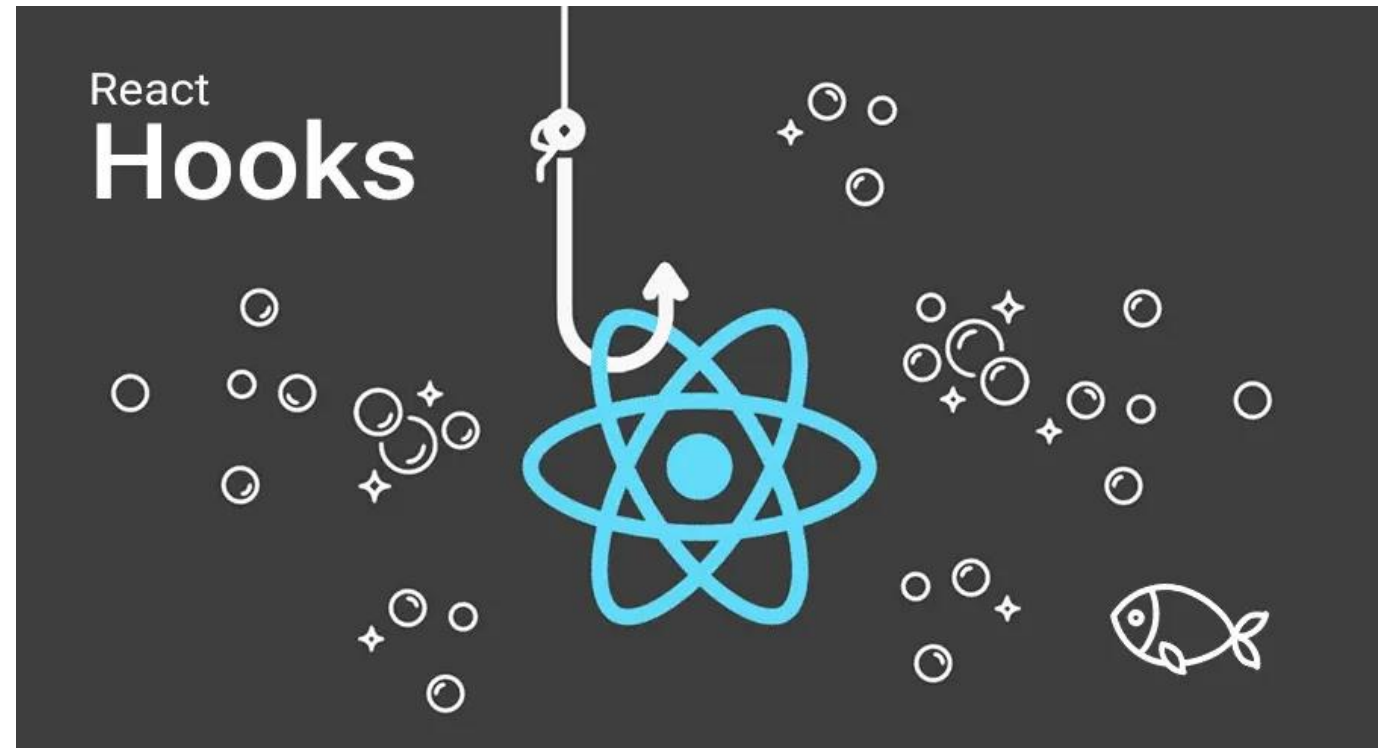
R. Naudts, J. Pieck, J. Rombouts, B. Van Impe

Hooks - reminder

Custom Hooks

Example hook

Hooks: reminder



What are hooks?

React Hooks:

Special functions that let you use state and other React features in functional components by “hooking” into the component lifecycle.

React and state

The core mental model of react is:

“The UI is a function of the state”

also known as **UI=f(state)**

How?

Automatic Re-renders: When a component's state changes, React automatically detects the change and re-renders only the necessary parts of the UI to reflect the new state value.

React and state

React determines when to re-render a component based *only* on specific triggers:

- A change in **state**
- A change in **props** (data passed from a parent component).
- A parent component re-rendering.

Local variables are fleeting they are not part of the state tracking mechanism: they don't trigger re-renders and are reset after a re-render.

useState hook revisited

The useState hook:

- persists the variable's value between renders (it stores the value outside the function's scope, managed internally by React).
- The setFunction acts as a signal to React that something important has changed and the component needs to be run again (re-rendered) with the new, persistent value.

Sample hooks in React

useState Allows you to add state to functional components.

useEffect Handles side effects in functional components.

useContext Accesses the value of a React context.

Sample hooks in React router & Next.js

useParams Accesses the dynamic parameters of the current route.

A **route** refers to a mapping between a URL path and a specific component in your application. It determines which component should render when a user navigates to a particular URL.



useRouter Provides access to the Next.js router object, allowing you to interact with the routing system (e.g., navigation, query parameters, pathnames).

third-party hooks

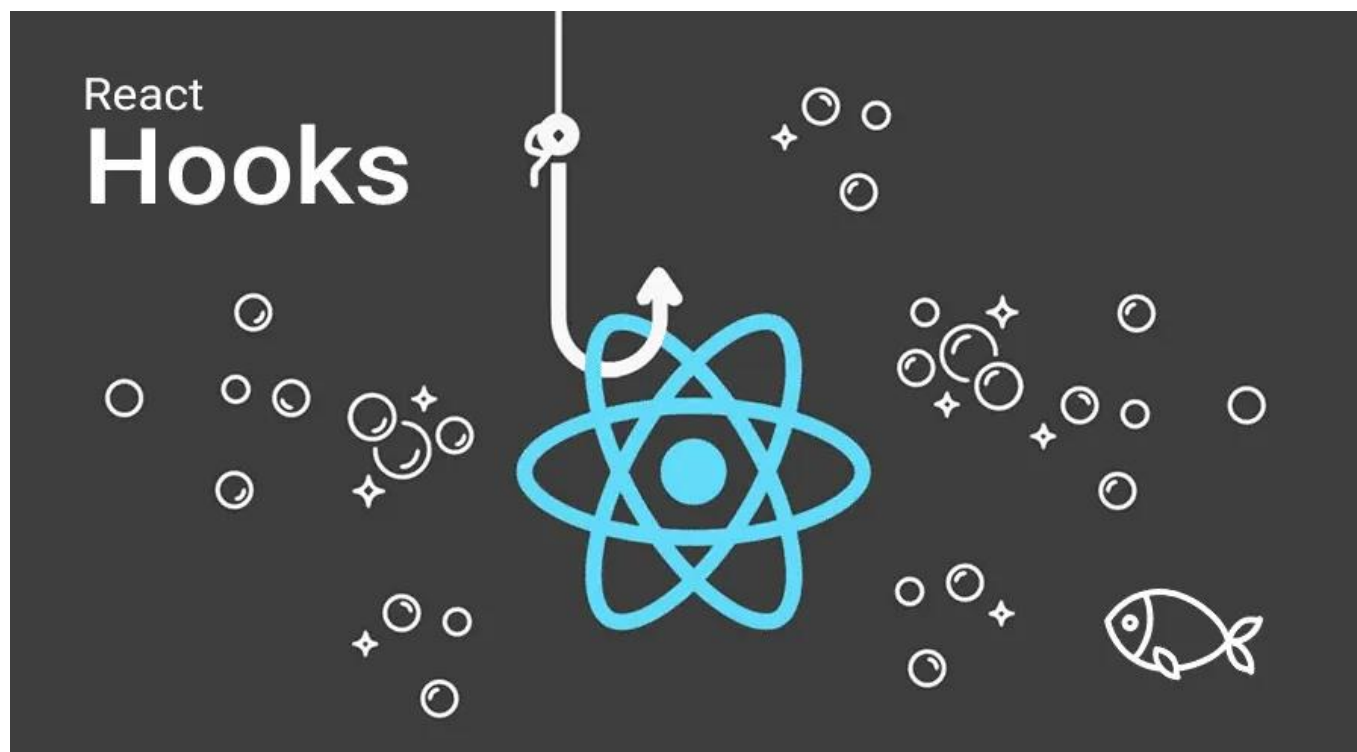
- **useInterval** encapsulates the logic for setting up and clearing intervals, making it safer and more reusable.
- **useSWR** Simplifies data fetching, caching, and revalidation in React applications.
- **useTranslation** Enables internationalization in React apps by providing translation functions and language management. (see slides about internationalisation)

Hooks - reminder

Custom Hooks

Example hook

Custom Hooks



Why make custom hooks

From the official documentation:

“Use custom hooks to share stateful logic between components”

* Stateful logic means behaviour that uses data.

Why make custom hooks

Share state or logic? Why not use:

A service?

- A state change in a service doesn't trigger the re-rendering of a component without manually calling it.
- Services can't use hooks internally

A child component?

- Hooks don't have a UI
- A component's main responsibility is rendering, not logic.

react context:

- There is only 1 global state in a context, each instance of a hook has their own state to work with

Why make custom hooks

Best practice:

- Use **custom hooks** for reusable **stateful logic** that depends on React features.
- Use **services** or **TypeScript functions** for **stateless logic** or non-React tasks.
- Use **React Context** for **global state** that needs to be shared across many components.
- Use **child components** for **UI composition** and encapsulating UI-based logic.

Rules for creating custom hooks

- **Hook names must start with use followed by a capital letter:**
 - useState (built-in) or
 - useOnlineStatus (custom).
- Hooks may return arbitrary values.
- Only custom hooks that internally call a built-in hook are considered hooks by react

```
// Not a hook: Starts with "use" but doesn't call any hooks
const useAdd = (a: number, b: number) => {
  return a + b; // Just a regular function
}
```

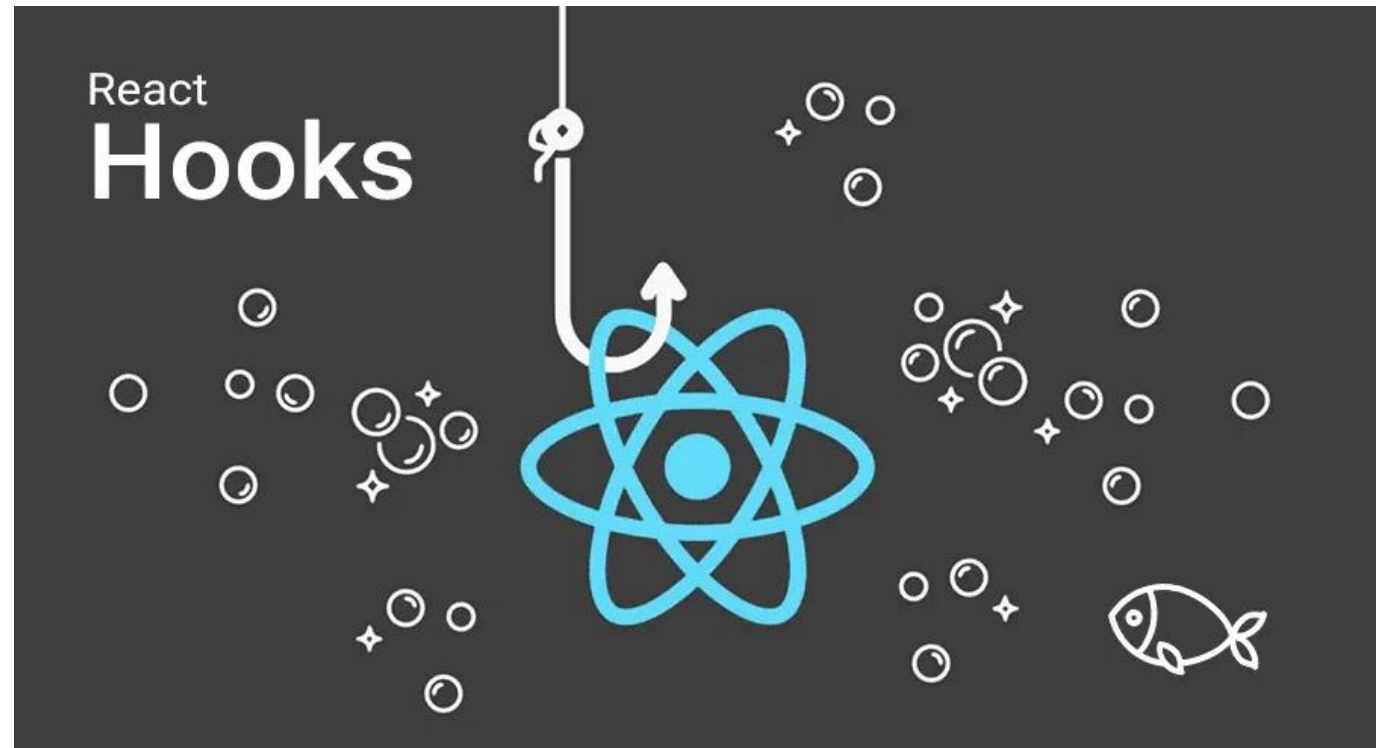
note that this means that most custom hooks can only be used in client components.

Hooks - reminder

Custom Hooks

Example hook

useCounter example



start code

useState and useEffect only work in client components

Here is the stateful logic of this component.

This is not reusable somewhere else.
What if we want more counters in
our application?

```
"use client";
```

```
import { useState, useEffect } from "react";
```

```
const Counter: React.FC = () => {  
  // Stateful logic: managing a counter  
  const [count, setCount] = useState(0);  
  
  // Stateful logic: side effect to log the count  
  useEffect(() => {  
    console.log(`Count is now: ${count}`);  
  }, [count]);  
}
```

```
return (  
  <div>  
    <p>You clicked {count} times</p>  
    <button  
      className="bg-blue-500 hover:bg-blue-700  
      onClick={() => setCount(count + 1)}  
    >  
      Click me  
    </button>  
    <button  
      className="bg-red-500 hover:bg-red-700  
      onClick={() => setCount(0)}  
    >  
      Reset  
    </button>  
  </div>  
)  
);  
;
```

Custom Hook

We use inner hook useState to keep the count and the setter for count

We create the increment and reset functions and return them, so they can be used in other components.

Returning the state and 2 methods like this allows for the normal destructuring syntax and hides setCount

```
import { useState } from "react";

const useCounter = (initialValue = 0) => {
  const [count, setCount] = useState(initialValue);

  const increment = () => setCount(count + 1);
  const reset = () => setCount(initialValue);

  return { count, increment, reset };
};

export default useCounter;
```

Using our hook

Using the hook, just like a built-in one.

We can use the exposed state and functions from this component.

We **cannot** use `setCount` directly, because we didn't return that.

```
"use client";    You, 2 days ago • Initial commit
import { useEffect } from "react";
import useCounter from "../hooks/useCounter";

const SingleCounter: React.FC = () => {
  const { count, increment, reset } = useCounter(0);

  useEffect(() => {
    console.log(`Count is now: ${count}`);
  }, [count]);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button
        className="bg-blue-500 hover:bg-blue-700 text-
        onClick={increment}
      >
        Click me
      </button>
      <button
        className="bg-red-500 hover:bg-red-700 text-wh
        onClick={reset}
      >
        Reset
      </button>
    </div>
  );
};
export default SingleCounter;
```

Hooks and state

- Hooks don't share state

```
const DoubleCounter: React.FC = () => {  
  // First counter instance  
  const { count: count1, increment: increment1, reset: reset1 } = useCounter(0);  
  // Second counter instance  
  const { count: count2, increment: increment2, reset: reset2 } = useCounter(0);  
  
  useEffect(() => {  
    console.log(`Counter 1: ${count1}, Counter 2: ${count2}`);  
  }, [count1, count2]);  
}
```

- This code has 2 counters, each with their own state and methods, we use a slightly different syntax to make the distinction.
- In contrast: Context has 1 shared state

Hooks and state

Counter 1

You clicked 7 times

Increment

Reset

Counter 2

Count: **3**

+ Add One

↺ Reset

